

EJERCICIOS DE AMPLIACIÓN

MF0227_3 · Programación Orientada a Objetos
Certificado de Profesionalidad IFCD0112

Catorce ejercicios que completan los tres cuadernos del módulo. Cubren los criterios de evaluación del Real Decreto 628/2013 que no tenían ejercicio asociado, y añaden dos contenidos que el Real Decreto no exige pero que conviene dominar al terminar: la entrada y salida de ficheros y los tipos genéricos propios.

Cada enunciado indica los criterios de evaluación que trabaja.
La solución comentada y ejecutable de cada uno está en el paquete **ampliacion** del proyecto Java.

ÍNDICE

Unidad formativa	Ejercicios	Nº
UF2404 — Principios de la programación orientada a objetos	6	A1 – A4, A13 – A14
UF2405 — Modelo de programación web y bases de datos	2	A5 – A6
UF2406 — El ciclo de vida del desarrollo de aplicaciones	6	A7 – A12

Criterios de corrección sugeridos: la solución debe compilar y ejecutarse sin errores (30%), cumplir todos los requisitos funcionales del enunciado (40%), aplicar correctamente los conceptos trabajados (20%) y estar documentada y con nombres de identificadores descriptivos (10%).

Los ejercicios A1 a A4 pueden plantearse en cualquier momento después del bloque 3 de la UF2404. Los A5 y A6 requieren haber completado el bloque 3 de la UF2405. Los A7 a A12 se apoyan en los ejercicios 9, 12 y 14 de la UF2406, así que van al final. Los A13 y A14 son de concepto y no necesitan nada previo: el A14 encaja bien nada más terminar la herencia, y el A13 conviene dejarlo para cuando el alumnado haya visto ya el cuaderno de la UF2406 completo, porque recorre sus artefactos.

UF2404 — PRINCIPIOS DE LA PROGRAMACIÓN ORIENTADA A OBJETOS

EJERCICIO A1. Ciclo de vida del objeto y gestión de memoria

Criterios de evaluación: CE2.2, CE1.2

En Java nadie destruye los objetos a mano: de eso se encarga el recolector de basura. Este ejercicio pretende que compruebes cuándo deja de existir realmente un objeto, y por qué un constructor tiene pareja pero un destructor no.

Se pide:

1. Escribe una clase **Documento** con un contador estático de instancias creadas, de modo que en cualquier momento se pueda saber cuántos objetos se han construido.
2. Crea tres documentos dentro de un método y comprueba el contador al entrar y al salir del método. Explica por qué el contador **no** baja.
3. Guarda una referencia débil (`WeakReference`) a uno de los documentos, elimina después todas las referencias normales que apunten a él, solicita la recolección y comprueba si la referencia débil se ha quedado vacía. Explica qué demuestra ese resultado.
4. Repite la prueba anterior pero guardando además el documento en una lista estática de la clase. Explica por qué ahora no se recolecta y qué nombre recibe ese problema.
5. Implementa una clase **Conexion** que represente un recurso externo, de forma que se libere de manera **determinista**. Explica en un comentario por qué `finalize()` no sirve para eso y qué se usa en su lugar.
6. Explica en un comentario la diferencia entre la memoria de la pila y la del montículo, e indica dónde vive el objeto y dónde la variable que lo referencia.

Pista. Solicitar la recolección no es lo mismo que provocarla. `System.gc()` es una sugerencia que la máquina virtual puede ignorar, así que la prueba del apartado 3 hay que plantearla admitiendo que a veces no concluya, y decirlo, en lugar de dar por hecho que el objeto ha muerto.

EJERCICIO A2. Estructuras de datos genéricas

Criterios de evaluación: CE2.7, CE2.9, CE2.8

La pila y la cola que implementaste en el bloque 3 de la UF2404 solo sirven para un tipo de dato. Si necesitas una pila de cadenas y otra de enteros, hay que escribir la clase dos veces. Este ejercicio te pide generalizarlas.

Se pide:

1. Reescribe la pila como una clase genérica **Pila<T>** que funcione con cualquier tipo, conservando las operaciones apilar, desapilar, cima, estaVacía y tamaño.
2. Haz lo mismo con la cola: **Cola<T>**, con encolar, desencolar, primero, estaVacía y tamaño.
3. Comprueba las dos con al menos tres tipos distintos, incluido uno propio, y explica qué ventaja concreta aporta frente a haber usado `Object` en lugar de `T`.
4. Implementa una **ColaConPrioridad<T>** que solo admita tipos ordenables, de modo que el compilador rechace un tipo que no lo sea. Investiga cómo se declara esa restricción.

5. Intenta crear dentro de la pila un array del tipo genérico (`new T[10]`). Explica por qué el compilador no te deja y cómo se resuelve en la práctica.
6. Añade a la pila un método **apilarTodos** que acepte cualquier colección cuyos elementos sean del tipo de la pila o de un subtipo suyo, y explica qué significa la notación que has tenido que emplear.
7. Explica en un comentario qué es el borrado de tipos y por qué, en tiempo de ejecución, una `Pila<String>` y una `Pila<Integer>` son la misma clase.

Pista. Los genéricos no existen en tiempo de ejecución. Todo lo que hacen ocurre durante la compilación, y esa es a la vez su mayor virtud —los errores aparecen antes de ejecutar— y la explicación de casi todas sus limitaciones.

EJERCICIO A3. Persistencia en ficheros

Criterios de evaluación: CE2.8, CE2.10, CE2.3

Todo lo que has programado hasta ahora desaparece al cerrar el programa. Antes de guardar datos en una base de datos (UF2405), conviene saber guardarlos en un fichero, que es la forma más sencilla de persistencia y la que se usa para configuraciones, registros e intercambio de datos.

El videoclub del ejercicio 13 de la UF2404 debe conservar su catálogo entre ejecuciones.

Se pide:

1. Implementa un método que guarde una lista de artículos en un fichero de texto con formato CSV, una línea por artículo, empleando las clases de la biblioteca estándar y garantizando que el fichero se cierra pase lo que pase.
2. Implementa el método inverso, que lea ese fichero y reconstruya la lista de objetos.
3. Comprueba el ciclo completo: guardar, leer y verificar que lo leído coincide exactamente con lo guardado.
4. Fija explícitamente la codificación de caracteres al escribir y al leer, y explica en un comentario qué ocurre si no se hace.
5. Gestiona los errores que pueden darse: que el fichero no exista, que una línea esté mal formada y que no haya permisos. Decide en cada caso si conviene abortar o continuar, y justifícalo.
6. Añade un método que registre una línea en un fichero de bitácora sin borrar lo anterior, e indica qué opción de apertura lo permite.
7. Guarda ahora la misma lista mediante serialización de objetos. Compara las dos soluciones e indica cuándo usarías cada una.
8. **Ampliación:** comprueba qué ocurre con el identificador de versión de serialización si se añade un atributo a la clase, y explica por qué conviene declararlo a mano.

Pista. Al leer un CSV, no basta con partir la línea por comas. Piensa qué pasa con una película titulada «Alien, el octavo pasajero».

EJERCICIO A4. Análisis de un programa ajeno

Criterios de evaluación: CE2.5, CE2.6, CE1.8

Programar es solo la mitad del oficio: la otra mitad es leer código que ha escrito otra persona. En un trabajo real se lee mucho más de lo que se escribe, y casi nunca desde el principio.

Se te entrega el código del ejercicio 13 de la UF2404, el sistema del videoclub, **sin su enunciado**. No tienes que modificarlo: tienes que entenderlo.

Se pide:

1. Localiza en el código cada una de las siguientes características del paradigma orientado a objetos, indicando clase y línea: encapsulación, herencia, polimorfismo, clase abstracta, método redefinido, composición, agregación y jerarquía de excepciones.
2. Reconstruye el diagrama de clases a partir del código, con sus relaciones y multiplicidades, sin mirar la documentación.
3. Identifica los métodos que forman la **interfaz** de cada clase (lo que se puede usar desde fuera) y sepáralos de los detalles internos. Explica qué criterio has seguido.
4. Deduce, solo leyendo el código, cuáles son las tres reglas de negocio del videoclub y en qué método está implementada cada una.
5. Utiliza las herramientas del entorno de desarrollo para responder más deprisa a los apartados anteriores. Documenta qué función has usado para cada cosa:
 - Ver la jerarquía de tipos de una clase.
 - Encontrar todos los usos de un método.
 - Saltar a la declaración de un símbolo y volver.
 - Ver la estructura de un fichero de un vistazo.
 - Generar el diagrama de clases, si tu entorno lo permite.
6. Escribe un informe de media página explicando qué hace el programa, dirigido a alguien que va a modificarlo la semana que viene.
7. **Ampliación:** escribe un programa que analice una clase por reflexión y responda automáticamente a parte del apartado 1.

Pista. Para entender código ajeno, empieza por los tipos y no por los algoritmos. Saber qué clases hay y cómo se relacionan explica el noventa por ciento; el cuerpo de los métodos casi siempre se deduce después.

EJERCICIO A13. El ciclo de desarrollo orientado a objetos

Criterios de evaluación: CE1.1, CE1.8

Este proyecto no empieza en la primera clase que escribiste: empieza mucho antes, con un cliente que cuenta lo que necesita. El sistema de gestión del gimnasio que recorre la UF2406 está terminado, y sus artefactos siguen ahí: requisitos, tarjetas CRC, diagramas, código, pruebas y manuales. Este ejercicio pide recorrer ese ciclo hacia atrás, sobre un caso ya resuelto, para situar dónde encaja exactamente la programación orientada a objetos.

Se pide:

1. Enumera las fases del ciclo de desarrollo del software bajo el paradigma orientado a objetos y describe, para cada una, qué entra, qué sale y qué documento o artefacto la representa.
2. Localiza dentro de este proyecto el artefacto concreto que corresponde a cada fase del sistema del gimnasio y anótalo junto a la fase. Si alguna fase no tiene artefacto, dilo.
3. Comprueba la **trazabilidad** de la cadena: verifica que todo lo que entra en una fase ha salido de alguna fase anterior, y señala en qué punto exacto del ciclo entra la programación orientada a objetos.
4. Explica por qué la POO es **una** fase del ciclo y no el ciclo entero: qué decisiones están ya tomadas antes de escribir la primera clase, y qué queda por hacer después de que el código

compile.

5. Compara este ciclo con el del paradigma estructurado: qué unidad recorre las fases en cada uno, y qué consecuencia tiene eso sobre el mantenimiento cuando cambia un requisito.
6. Explica por qué el ciclo orientado a objetos se recorre en espiral y no en cascada, y qué significa que las fases compartan vocabulario.
7. Escribe un programa que, dado el nombre de una fase, devuelva sus entradas, sus salidas y su artefacto, y que compruebe de forma automática la trazabilidad de la cadena completa.

Pista. La prueba de si has entendido el ciclo es sencilla. Coge el nombre de una clase cualquiera del proyecto, por ejemplo **ClaseColectiva**, y rastrea hacia atrás de dónde ha salido: de un diagrama, que salió de unas tarjetas CRC, que salieron de un texto de requisitos, que salió de una conversación. Si el rastro se corta en algún punto, ahí falta una fase.

EJERCICIO A14. El paso de mensajes

Criterios de evaluación: CE1.4, CE2.4

En el paradigma orientado a objetos los objetos no se leen ni se manipulan: se les pide algo. A esa petición se le llama **mensaje**, y el comportamiento del programa es el resultado de decidir, en cada envío, quién responde. Este ejercicio pretende que veas esa decisión ocurrir.

Se pide:

1. Explica qué es un mensaje y de qué tres partes se compone. Relaciona cada parte con lo que escribes en Java al invocar un método.
2. Escribe una jerarquía **Notificador** con tres subclases (Email, Sms y Push) que respondan al mismo mensaje de formas distintas. Envía el mismo mensaje a los tres a través de una variable del tipo padre y anota qué responde cada uno.
3. Demuestra la diferencia entre **sobrecarga** y **redefinición**: construye un caso en el que el método elegido dependa del tipo declarado de la variable y otro en el que dependa del tipo real del objeto. Explica en qué momento se decide cada uno.
4. Comprueba tu respuesta con reflexión: para cada envío imprime la clase real del receptor, el selector, los tipos de los argumentos y la clase en la que se ha encontrado finalmente el método.
5. Envía un mensaje a una referencia nula, captura el error y explica por qué es un error de ejecución y no de compilación.
6. Usa **super** para enviar el mismo mensaje empezando la búsqueda en la superclase, y demuestra que el receptor sigue siendo el mismo objeto.
7. Explica por qué se dice que el comportamiento de un objeto es el conjunto de mensajes que sabe responder, y no el valor de sus atributos.

Pista. Para el apartado 3, prueba a declarar una variable de tipo **Object** que contenga una cadena y pásarsela a un método sobrecargado que tenga una versión para **Object** y otra para **String**. El resultado sorprende, y sorprende porque la sobrecarga la resuelve el compilador, que solo conoce lo que has escrito en la declaración.

UF2405 — MODELO DE PROGRAMACIÓN WEB Y BASES DE DATOS

EJERCICIO A5. Protocolos y formatos de intercambio

Criterios de evaluación: CE1.2, CE1.1

Has programado Servlets sin ver nunca lo que viaja de verdad entre el navegador y el servidor. Este ejercicio levanta la tapa: HTTP es texto plano sobre una conexión TCP, y se puede escribir a mano.

Se pide:

1. Escribe un servidor mínimo que escuche en un puerto, acepte una conexión TCP y responda con una respuesta HTTP correcta. No uses ningún contenedor web: solo sockets.
2. Escribe un cliente que envíe una petición GET escrita a mano y muestre por pantalla, línea a línea, lo que envía y lo que recibe.
3. Identifica en esa conversación las cuatro partes de una petición (método, ruta, versión y cabeceras) y las de una respuesta (versión, código de estado, cabeceras, línea en blanco y cuerpo).
4. Repite el intercambio con una petición POST que lleve datos en el cuerpo. Explica qué cabecera indica dónde termina ese cuerpo y qué pasaría si faltara.
5. Provoca una respuesta 404 y otra 302. Indica qué hace el navegador con cada código y relaciona el 302 con el patrón POST-Redirect-GET del ejercicio 11.
6. Analiza un documento XML con las clases del propio JDK, extrayendo los datos de una lista de mascotas.
7. Analiza un mensaje SOAP, que no es más que un XML con una estructura convenida, e identifica su sobre, su cabecera y su cuerpo.
8. Genera la misma información en JSON y compara los tres formatos: cuándo usarías cada uno.

Pista. HTTP separa las cabeceras del cuerpo con una **línea en blanco**. Ese detalle, que parece trivial, es lo que permite al receptor saber dónde dejan de ser metadatos y empiezan a ser datos.

EJERCICIO A6. Tecnologías de acceso a datos

Criterios de evaluación: CE2.2, CE2.1, CE1.3

En el bloque 3 has usado JDBC con DriverManager porque era lo más sencillo. En un proyecto real hay que **elegir** la tecnología de acceso a datos, y esa elección tiene consecuencias que se pagan durante años.

Se pide:

1. Enumera las tecnologías habituales de conexión y acceso a datos desde una aplicación Java, indicando de cada una qué problema resuelve: DriverManager, DataSource con pool, JPA/Hibernate y los llamados mapeadores ligeros.
2. Implementa tres estrategias de acceso con el **mismo contrato**, de forma que se puedan intercambiar sin tocar el resto del programa: una que abra conexión en cada operación, otra que reutilice una del pool y otra que además convierta automáticamente las filas en objetos.
3. Cronometra las tres haciendo el mismo trabajo y comenta los resultados. Indica qué parte del coste es de la conexión y cuál del mapeo.

4. Cuenta las líneas de código que hace falta escribir con cada una para leer una entidad de cinco columnas, y compara ese dato con el anterior. Explica por qué la tecnología más rápida no siempre es la que conviene.
5. Elabora una tabla de decisión que indique, para cuatro escenarios concretos, cuál elegirías y por qué: una aplicación de escritorio de un solo usuario, una web con 200 usuarios simultáneos, un proceso nocturno que carga un millón de filas y un microservicio con dos tablas.
6. Explica qué aporta un DataSource frente a DriverManager más allá del rendimiento.
7. Compara brevemente el modelo de acceso a datos de la plataforma Java con el de .NET, señalando los elementos equivalentes.

Pista. La pregunta correcta no es cuál es la tecnología mejor, sino cuál es el problema dominante en este proyecto. Si el cuello de botella es el número de conexiones, la respuesta es un pool; si es la cantidad de código repetitivo, un mapeador; y si es el control fino de la consulta, JDBC.

UF2406 — EL CICLO DE VIDA DEL DESARROLLO DE APLICACIONES

EJERCICIO A7. Plantilla y documento de diseño de una clase

Criterios de evaluación: CE3.3, CE3.7

El Javadoc del ejercicio 11 documenta el código **ya escrito**. El documento de diseño es lo contrario: se escribe **antes**, y sirve para que alguien pueda programar la clase sin volver a preguntar.

Se pide:

1. Propón el índice (plantilla) del documento de diseño de una clase, explicando qué contiene cada apartado y por qué hace falta.
2. Justifica cada apartado respondiendo a esta pregunta: ¿qué decisión tendría que tomar por su cuenta quien programe la clase si ese apartado no estuviera?
3. Elabora el documento completo para la clase **GestorInscripciones** del sistema del gimnasio, siguiendo tu propia plantilla.
4. Escribe un comprobador que verifique que un documento concreto contiene todos los apartados obligatorios de la plantilla y señale los que faltan.
5. Explica en qué se diferencia este documento del Javadoc y del manual de usuario: destinatario, momento en que se escribe y qué pasa si no existe.
6. Indica qué apartados de tu plantilla quedan obsoletos en cuanto el código cambia, y propón cómo evitar que el documento acabe mintiendo.

Pista. Un documento de diseño que solo repite los nombres de los métodos no sirve para nada, porque eso ya lo dice el código. Lo que aporta valor es todo lo que el código **no** puede decir: por qué se eligió ese diseño, qué alternativas se descartaron y qué pasa en los casos raros.

EJERCICIO A8. Plantilla e informe de pruebas

Criterios de evaluación: CE3.4, CE3.8

Has diseñado pruebas (ejercicio 7), las has automatizado (ejercicio 9) y has ejecutado una regresión (ejercicio 12). Falta lo que se entrega al cliente o al responsable de calidad: el **informe**.

Se pide:

1. Propón el índice (plantilla) del documento de pruebas, que debe recoger tanto el **diseño** de las pruebas como sus **resultados**.
2. Explica por qué esos dos contenidos van en el mismo documento y qué pasaría si se entregaran por separado.
3. Elabora el informe completo de las pruebas del sistema del gimnasio, con los datos reales obtenidos al ejecutar las baterías de los ejercicios 9 y 12.
4. Genera el informe de forma **automática** a partir de la ejecución, en lugar de copiar los números a mano. Explica qué ventaja tiene.
5. Incluye en el informe el **veredicto**: se acepta la entrega o no, y con qué criterio se decide. Un informe que no concluye nada no sirve.
6. Añade el apartado de defectos detectados, con su gravedad y su estado, y explica la diferencia entre un defecto **abierto** y uno **aceptado**.

7. Indica qué cifras del informe **no** deben usarse como objetivo, y por qué.

Pista. Un informe de pruebas que solo diga «todas las pruebas pasan» es inútil, porque no dice cuántas eran ni qué cubrían. Cien pruebas que pasan sobre el diez por ciento del código son peores que veinte sobre el ochenta.

EJERCICIO A9. Manual de operación y mantenimiento

Criterios de evaluación: CE3.5, CE3.9

El sistema del gimnasio se entrega y alguien tiene que **mantenerlo funcionando**: instalarlo, actualizarlo, hacer copias de seguridad y resolver la llamada del sábado por la tarde cuando no arranca. Esa persona no eres tú, no ha visto el código y probablemente no sepa Java.

Se pide:

1. Propón el índice (plantilla) del manual de operación y mantenimiento, también llamado manual técnico.
2. Explica a quién va dirigido y en qué se diferencia del manual de usuario y del Javadoc. Los tres son documentación y ninguno sustituye a los otros dos.
3. Elabora el manual completo del sistema del gimnasio siguiendo tu plantilla.
4. Redacta con detalle el apartado de **resolución de incidencias**, con al menos cinco síntomas reales, su causa probable y su solución.
5. Redacta el apartado de **copias de seguridad** indicando qué se copia, cada cuánto, dónde y, sobre todo, cómo se comprueba que la copia sirve.
6. Escribe el **procedimiento de vuelta atrás**: qué hacer si una versión nueva sale mal en producción un viernes.
7. Explica qué información **no** debe aparecer nunca en este manual.

Pista. Escribe cada procedimiento como si lo fuera a seguir alguien a las tres de la madrugada, con prisa y sin poder preguntarte. Eso descarta las frases del tipo «configurar adecuadamente el servidor».

EJERCICIO A10. Gestión de la configuración del software

Criterios de evaluación: CE4.2, CE4.4, CE4.6

En el ejercicio 12 modificaste una clase siguiendo un procedimiento. Este ejercicio se ocupa de lo que rodea a ese cambio: cómo se identifica cada versión, cómo se guarda su historia y cómo se sabe exactamente qué código hay instalado en el gimnasio.

Se pide:

1. Explica qué es la gestión de la configuración del **software** y en qué se diferencia de la gestión de la configuración de la **documentación**.
2. Define los elementos de configuración del proyecto del gimnasio: qué se versiona y qué no, justificando cada exclusión.
3. Implementa un versionador que, dado el tipo de cambio, calcule la versión siguiente según el versionado semántico.
4. Escribe un validador de mensajes de registro de cambios que compruebe una convención acordada por el equipo, y pásale mensajes reales, buenos y malos.

5. Explica qué es una **línea base** y por qué hace falta poder reconstruir exactamente la versión que hay instalada en el cliente.
6. Aplica el procedimiento de cambio a un **documento**, no a una clase: indica cómo se identifica una edición y una revisión, y quién aprueba cada cambio.
7. Enumera las órdenes concretas del sistema de control de versiones para: crear la rama del cambio del ejercicio 12, integrarla, etiquetar la entrega y volver atrás si sale mal.

Pista. La pregunta que resuelve la gestión de la configuración es siempre la misma, y es más práctica de lo que parece: si el gimnasio llama mañana diciendo que algo falla, ¿puedes reconstruir **exactamente** el código que tienen instalado?

EJERCICIO A11. De la documentación de diseño a la interfaz

Criterios de evaluación: CE5.4, CE5.6, CE5.7

En el ejercicio 14 construiste una ventana a partir de un enunciado. En un proyecto real se construye a partir de la **documentación de diseño**, que llega ya hecha, y hay que respetarla.

Se te entrega el diseño de la pantalla de inscripción del gimnasio, con su distribución, sus componentes y su comportamiento.

Se pide:

1. Enumera las clases de la biblioteca de interfaz que necesitas para cada elemento del diseño, y explica qué criterio has seguido para elegir entre las que hacen cosas parecidas: por ejemplo, una lista desplegable frente a un grupo de botones de opción.
2. Construye la ventana respetando exactamente la distribución indicada, de modo que se comporte bien al cambiar de tamaño y al cambiar el tamaño de letra del sistema.
3. Incorpora recursos multimedia: el logotipo del gimnasio en la cabecera y un aviso sonoro cuando la inscripción falla.
4. Genera el logotipo mediante código en lugar de cargarlo de un fichero, y explica en qué casos conviene cada opción.
5. Programa las clases que conectan la interfaz con la aplicación, de forma que la ventana no contenga **ninguna** regla de negocio.
6. Demuestra esa separación: sustituye la capa de negocio por otra distinta sin tocar ni una línea de la ventana.
7. Explica qué problema tienen los recursos multimedia cargados desde una ruta absoluta, y cómo se resuelve al empaquetar la aplicación.

Pista. Si para probar un cambio en la lógica tienes que abrir la ventana y hacer clic, la separación no está bien hecha. La prueba definitiva es poder ejecutar la lógica entera sin que aparezca nada en pantalla.

EJERCICIO A12. Validación y accesibilidad de la interfaz

Criterios de evaluación: CE5.5, CE5.8

La pantalla de inscripción ya está construida. Ahora hay que **comprobar** que cumple lo que se pedía, y ese es un trabajo distinto de programarla.

El Real Decreto que regula este certificado menciona expresamente los criterios de accesibilidad para personas con discapacidad, así que no es un extra opcional: forma parte de lo que hay que verificar.

Se pide:

1. Establece los criterios de validación de la pantalla: qué se considera correcto y cómo se comprueba cada cosa objetivamente.
2. Comprueba que los formatos de entrada y salida son los esperados, incluyendo los casos raros: números con coma, fechas, campos vacíos y textos muy largos.
3. Clasifica los tipos de error que puede cometer el usuario en esta pantalla y decide cómo responde la interfaz a cada uno.
4. Mide el **contraste** entre el texto y su fondo para todos los mensajes de la pantalla, y comprueba si alcanzan el nivel exigido. Investiga qué relación de contraste se considera suficiente.
5. Comprueba que la pantalla se puede usar **entera sin ratón**, e indica qué hay que añadir al código para que así sea.
6. Revisa si la información se transmite únicamente mediante el color, y corrígelo si es el caso.
7. Redacta el índice de la guía de usuario de esta pantalla.
8. Elabora el informe de validación con el veredicto y la lista de defectos encontrados.

Pista. Para el apartado 6, imprime la pantalla en blanco y negro. Todo lo que deje de distinguirse es información que estabas transmitiendo solo con el color, y que no llega a una de cada doce personas.

COBERTURA DE LOS CRITERIOS DE EVALUACIÓN

Los tres cuadernos del módulo cubren con ejercicio 47 de los 68 criterios de evaluación del Real Decreto. Con estos catorce ejercicios la cobertura pasa a 68.

Unidad formativa	Antes	Después	Total
UF2404	13	18	18
UF2405	11	14	14
UF2406	23	36	36
TOTAL	47	68	68

Con los ejercicios A13 y A14 no queda ningún criterio del módulo sin ejercicio asociado. Los dos últimos cierran los dos criterios de la UF2404 que empiezan por «explicar» —el ciclo de desarrollo y el paso de mensajes—, que hasta ahora solo se trataban en la teoría; y el CE4.5 de la UF2406 lo cumple punto por punto el ejercicio 12 de su cuaderno, que ahora lo declara junto a CE4.1 y CE4.3. Conviene reflejar ese último cambio también en el cuaderno original en .docx.

Referencia normativa: Real Decreto 628/2013, de 2 de agosto (BOE núm. 225, de 19 de septiembre de 2013), anexo IV, certificado IFCD0112. El PDF oficial tiene varios párrafos con la codificación de fuente dañada; el texto de esos criterios se ha recuperado glifo a glifo y contrastado con el resto del articulado, y la correspondencia completa está en recursos/MATRIZ_CE.md.